

DAY 07: KAFKA FROM THE CLI

Data Bytes Technologies · Java 25 & Modern Stack

Yesterday you talked to **PostgreSQL** with `psql`, from a shell **inside its container**. Today we meet a second stateful service (**Apache Kafka**) and drive it the very same way: by hand, with its own command-line tools, from inside the container. No app, no framework. Just Kafka.

// WHERE WE ARE

- **Day 6: PostgreSQL.** A stateful service we ran in Docker and drove with `psql`, from a shell **inside the container**:
`docker compose exec postgres psql ...`.
- **Days 3–5: Java.** The language and concurrency model the week-2 app will be written in.
- **Days 1–2: Git & Docker.** The repo and the one-command environment everything runs on.

💡 Kafka is **another stateful service we drive by its own CLI inside Docker**, same move as `psql`, different tool. Today is the **mental model** plus the **tools**; the *code* comes in week 2.

// BY THE END OF TODAY, YOU CAN...

- explain **why** events decouple services, and when a plain call is the better choice
- describe a **topic**, **partition**, **consumer group**, and **offset** in your own words
- **create**, **list**, and **describe** a topic from the Kafka CLI inside the container
- **produce** messages by hand and **consume** them, and prove a topic is a *log*, not a queue
- watch a **key** pick a partition, inspect a group's **lag**, and trigger a **rebalance**

// A CALL VERSUS AN EVENT

So far every request did all its work before replying. What if registering an applicant should *also* send a welcome email, update analytics, and warm a cache?

- **A call:** the service phones each one and waits. If one is slow or down, the registration is too.
- **An event:** the service says "an applicant registered" *once*, and moves on. Whoever cares listens.

The producer records that **a fact happened**. It doesn't know or care who reacts.

// WHY EVENTS DECOUPLE: IN PLAIN TERMS

💡 **In plain terms:** a synchronous call couples services **in time**: the caller is only as healthy as the callee. An event breaks that link: the producer writes the fact and is done; consumers react on **their own** schedule.

The payoff: you can add a *new* reaction (say, fraud scoring) by adding a **new consumer**. The original service never changes. The system grows **sideways**.

// ...AND WHEN *NOTTO*

Events are not free. You pay with:

- **eventual consistency**: the reaction happens *soon*, not in the same instant
- **a broker to operate**: one more stateful service to run and monitor (that's today's skill)
- **harder debugging**: the flow is no longer one straight line you can step through

💡 **In plain terms**: reach for events when **several independent things** must react to a fact. For a single request that needs an **immediate answer**, just make the call.

// MEET KAFKA

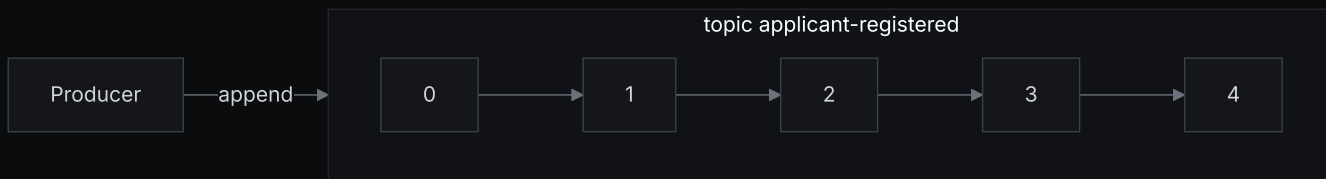
The broker we'll drive today is **Apache Kafka**. Before any tooling, four words carry the whole model:

- **topic**: a named, append-only log of messages
- **partition**: a slice of a topic, for parallelism
- **consumer group**: a team of readers sharing the work
- **offset**: a bookmark: how far a reader has got

Learn these four and Kafka stops being magic. We take them one at a time. Then we'll go and *touch* each one from the command line.

// A TOPIC IS A LOG, NOT A QUEUE

💡 In plain terms: a **topic** is an **append-only log**. Producers append to the end; reading a message does **not** remove it. Many readers can read the same log independently.



That "reading doesn't remove" property is *why* you'll be able to start a fresh consumer later and **replay the whole history** from position 0, something you will literally do today.

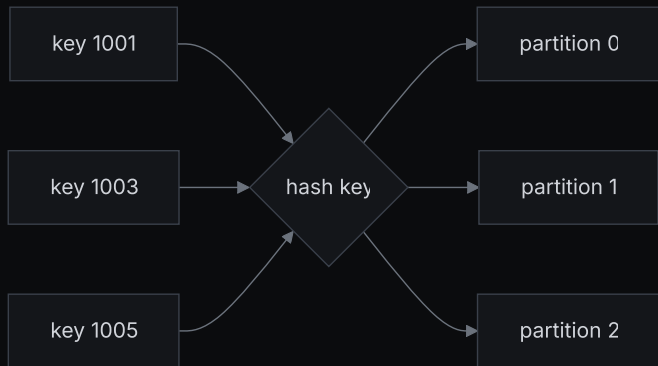
// PARTITIONS: PARALLELISM WITH ORDER KEPT

A busy topic is split into **partitions** so many consumers can read at once. But you still want all events for **one applicant** to stay in sequence.

💡 **In plain terms:** each message can carry a **key** (we'll use the **applicant id**). Kafka sends the same key to the same partition, so one applicant's events keep their order, while *different* applicants spread across partitions for throughput.

Ordering is guaranteed **within a partition**, never across the whole topic. That trade is the heart of Kafka's scale.

// ONE PICTURE: A KEY PICKS A PARTITION



Same key → same partition, **always**. Different keys fan out. No key? Kafka spreads messages round-robin, with no ordering guarantee between them.

// WHAT A MESSAGE CARRIES

A Kafka message is small. Most of it is data you choose; a little is bookkeeping the broker adds.

- **key** (*optional*): what you choose; it picks the partition (the applicant id, for us)
- **value**: the payload, just bytes (we'll type plain text today, JSON later)
- **partition + offset**: assigned by the broker; together they are the message's address
- **timestamp**: when it was produced

You'll print the **key** and **partition** when you consume. That's how you'll see the key choosing a partition.

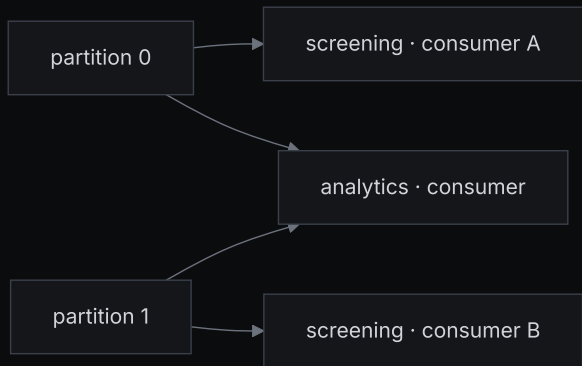
// CONSUMER GROUPS: SHARE THE WORK

Run three copies of a consumer and you don't want all three handling the *same* message.

A **consumer group** shares a topic's partitions among its members. Add an instance and Kafka **rebalances**: each partition is handled by exactly one member of the group. Scale by adding instances; no code change.

Two *different* groups, though, each get **every** message. That's how Screening and, later, Analytics can both react to the same applicants, independently.

// ONE PICTURE: GROUPS DIVIDE; GROUPS DUPLICATE



Inside **screening**, A and B split the partitions, each message handled **once**. The separate **analytics** group reads **everything**. Same log, two independent readers.

// OFFSETS: EACH GROUP HAS ITS OWN BOOKMARK

💡 **In plain terms:** an **offset** is a per-group **bookmark** into each partition. The group **commits** "I've processed up to here." Restart, and it resumes from the bookmark, not the beginning.

Because the bookmark belongs to the *group*, a brand-new group can start its bookmark at **zero** and **replay the entire history**. Same log, two different reading positions, and **lag** is just "how far the bookmark is behind the tail."

// A CONSUMER'S LOOP: POLL, PROCESS, COMMIT

A consumer isn't pushed to; it **pulls** in a loop. Each turn does three things.

- **poll**: ask the broker "anything after my bookmark?" and get a batch
- **process**: do your work with those messages
- **commit**: move the bookmark forward: "I'm done up to here"

Commit *after* processing, and a crash just means re-reading a few messages, never losing them. That commit is exactly what `kafka-consumer-groups.sh` reads back as `CURRENT-OFFSET`.

// RETENTION: THE LOG ISN'T KEPT FOREVER

A log that only grows would fill every disk. Kafka **ages messages out** on a policy you set per topic.

- **Time-based:** keep messages for, say, 7 days (`retention.ms`), then delete the old segments.
- **Size-based:** keep at most N bytes per partition (`retention.bytes`).
- **Compaction:** an alternative that keeps only the *latest* message per key (think "current state").

💡 **In plain terms:** retention is the dial between "a short buffer" and "the system of record." Replay only reaches back as far as retention has kept.

// AUTOMQ: KAFKA, WITH THE STORAGE RETHOUGHT

We run **vanilla Apache Kafka** locally because it's the simplest way to start (`apache/kafka` in our Compose file).

💡 In plain terms: **AutoMQ** is a drop-in, **Kafka-API-compatible** broker that offloads its log to **object storage (S3)** instead of local disks, cheaper and elastic in the cloud, with **no client changes**.

Every command you run today works against AutoMQ **unchanged**. Only the broker deployment differs. The four words you just learned are still the whole model.

// HOW WE'LL DRIVE KAFKA TODAY

Same move as `psql` yesterday: open a shell **inside the broker's container** and run its CLI.

```
# the pattern for every command today (Compose V2):  
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

- the tools live at `/opt/kafka/bin/...` in the `apache/kafka` image (full path; they're not on `PATH`)
- `--bootstrap-server localhost:9092` is the broker **as seen from inside its own container**
- four tools: `kafka-topics.sh`, `kafka-console-producer.sh`, `kafka-console-consumer.sh`, `kafka-consumer-groups.sh`

// FOUR WORDS, FOUR TOOLS

Each concept you just learned has exactly one CLI tool that touches it.

- **topic** (and its partitions) → `kafka-topics.sh` : create, list, describe
- **producer / keys** → `kafka-console-producer.sh` : append messages
- **consumer / log** → `kafka-console-consumer.sh` : read, replay
- **group / offset / lag** → `kafka-consumer-groups.sh` : inspect bookmarks

That's the whole toolbox. Everything for the rest of the day is one of these four, run with the pattern from the last slide.

// TRY IT NOW (3 MIN): BRING IT UP, CREATE A TOPIC

Start the infrastructure, then create your first topic with **3 partitions**:

```
docker compose -f infra/compose.yaml up -d postgres kafka kafka-ui # starts Postgres + Kafka + Kafka UI

docker compose -f infra/compose.yaml exec kafka \
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic applicant-registered --partitions 3 --replication-factor 1
# → Created topic applicant-registered.
```

`--partitions 3` is the parallelism dial from earlier; `--replication-factor 1` is fine for one broker on a laptop (you'd raise it in production for fault tolerance).

// TRY IT NOW (2 MIN): LIST AND DESCRIBE

See your topic, then look inside it:

```
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list  
  
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 \  
  --describe --topic applicant-registered
```

`--describe` prints **one line per partition** (0, 1, 2) with its leader and replicas. That's the "slice of the log" idea from the diagram, made real.

// TRY IT NOW (4 MIN): PRODUCE MESSAGES BY HAND

Open a producer with **keys**. Type lines as `key:value`, then **Ctrl-D** to finish:

```
docker compose -f infra/compose.yaml exec -it kafka \
/opt/kafka/bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
--topic applicant-registered --property parse.key=true --property key.separator=:
```

```
1001:applicant 1001 registered
1003:applicant 1003 registered
1005:applicant 1005 registered
1001:applicant 1001 cleared
```

You're appending to the log by hand. The **key** is the applicant id. Note you sent **1001 twice**; we'll prove both copies landed in the **same** partition.

// TRY IT NOW (3 MIN): CONSUME FROM THE BEGINNING

Read the whole log, showing each message's **partition** and **key**. **Ctrl-C** to stop:

```
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \  
  --topic applicant-registered --from-beginning \  
  --property print.partition=true --property print.key=true
```

Both **1001** lines share one partition; **1003** and **1005** sit elsewhere. The key picked the partition. Now run the **exact same command again**: every message **reappears**. A log, not a queue.

// 🖐️ EVERYONE WITH ME?

You should, right now, have:

- a topic `applicant-registered` with 3 partitions (`--describe` shows three rows)
- messages you **produced by hand**, visible when you **consume** `--from-beginning`
- proof it's a **log**: re-running the consumer shows the **same** messages again
- keyed messages where **the same key** sits in **the same partition**

If any of those is missing, flag it now. The group, lag, and rebalance exercises build directly on this.

// READING AS A GROUP: BOOKMARKS APPEAR

An anonymous consumer keeps no bookmark. Add `--group` and Kafka starts tracking offsets for it.

```
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \  
  --topic applicant-registered --from-beginning --group screening
```

- the first time, it reads from **0** and then **commits** how far it got
- stop it (Ctrl-C) and start it **without** `--from-beginning`. It resumes from the **bookmark**, not 0
- a *different* `--group analytics` starts at 0 again, its own bookmark, the whole history

// TRY IT NOW (3 MIN): INSPECT A GROUP'S LAG

List groups, then read the bookmark and lag for **screening**:

```
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list  
  
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \  
  --describe --group screening
```

Read the columns: **CURRENT-OFFSET** (the bookmark) vs **LOG-END-OFFSET** (the tail). **LAG** is the gap. Produce a few more messages and re-run. Watch **LAG** rise, then fall as the group catches up.

// TRY IT NOW (3 MIN): A FRESH GROUP REPLAYS EVERYTHING

Start a **brand-new** group and watch it read the whole history from 0:

```
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \  
  --topic applicant-registered --from-beginning --group analytics
```

Stop it (Ctrl-C), then describe the new group's bookmarks:

```
docker compose -f infra/compose.yaml exec kafka \  
  /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \  
  --describe --group analytics
```

`screening` and `analytics` read the **same** log at their **own** positions, the replay superpower from the offsets slide, in your terminal.

// TRY IT NOW (4 MIN): WATCH A REBALANCE

In **two terminals**, start two consumers in the **same group**:

```
# terminal 1 AND terminal 2, identical command:
docker compose -f infra/compose.yaml exec kafka \
  /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic applicant-registered --group workers
```

Then, in a **third terminal**, produce several keyed messages and watch where they appear:

```
docker compose -f infra/compose.yaml exec -it kafka \
  /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server localhost:9092 \
  --topic applicant-registered --property parse.key=true --property key.separator=:
```

Kafka splits the 3 partitions across the two members. Each message arrives in **one** consumer, not both. Kill one (Ctrl-C) and the survivor **rebalances** to take over all partitions.

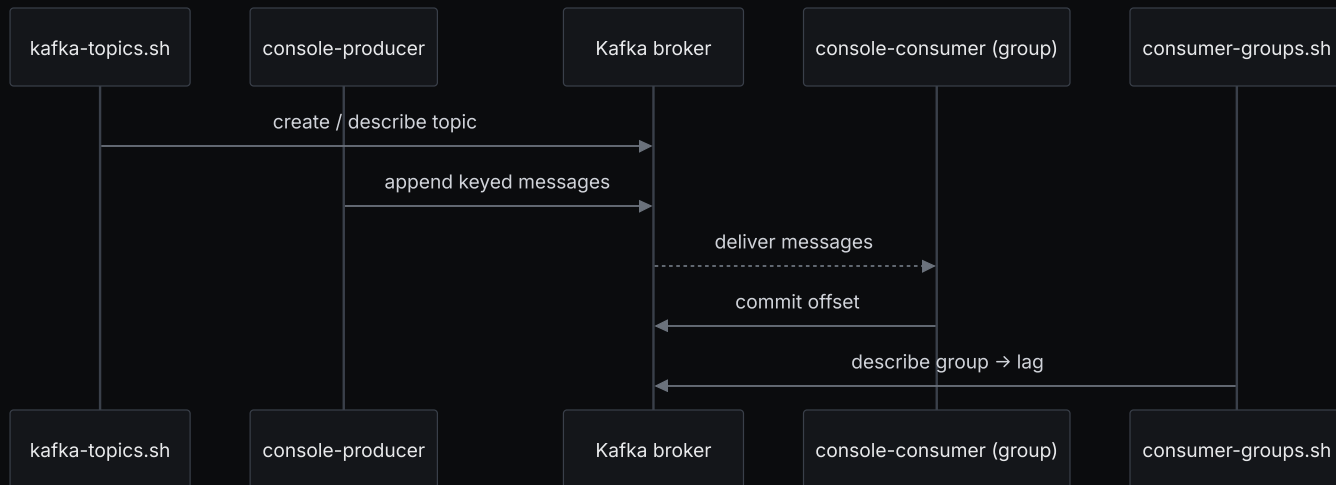
// THE KAFKA UI: A VISUAL COMPANION

Everything you did at the CLI is also visible in a browser, at <http://localhost:8081>.

- **Topics** → `applicant-registered` → its **partitions**, message counts, and the **messages** themselves
- **Consumers** → groups like `screening` and `workers`, their members and **lag** per partition
- a fast way to see what `--describe` told you in text, and to eyeball message **keys** and values

 Open it now: find your `applicant-registered` topic, click into a partition, and read a message you produced, key and all.

// PUTTING IT TOGETHER: ONE PICTURE



Four small tools, one broker. This is the entire shape of what we did, and exactly what an *application* will do in week 2, just from code instead of your keyboard.

// COMMON TRAPS

- **Treating a topic like a queue:** reading doesn't delete. Re-run the consumer `--from-beginning` and the messages are still there; many groups can read independently.
- **Expecting global ordering:** order holds only **within a partition**. The key is what keeps one applicant's events together.
- `kafka-topics.sh: not found`: the scripts aren't on `PATH`; use the full path `/opt/kafka/bin/...`.
- **Forgetting** `--from-beginning` **for a new look:** a named group resumes from its **bookmark**, so it may show *nothing* if it already read to the tail.

// RECAP, IN PLAIN TERMS

Today Kafka stopped being a black box. You drove it by hand:

- a **topic** is an append-only **log**; **partitions** parallelise, a **key** preserves per-key order
- a **consumer group** shares the work; an **offset** is each group's bookmark, **lag** the gap to the tail
- four CLI tools (**topics**, **producer**, **consumer**, **consumer-groups**) run from **inside the container**, just like `psql`
- the **Kafka UI** shows the same partitions, offsets, and messages visually

You created, produced, consumed, replayed, measured lag, and triggered a rebalance, all without a single line of application code.

// TODAY'S LAB

 **Drive Kafka entirely from the CLI:** create a partitioned topic, produce keyed messages, consume and **replay** them, read a consumer group's **lag**, and trigger a **rebalance** with two consumers. The Kafka UI is your visual cross-check.

Brief: `labs/day-07/` · You practise against the **real broker** from `docker compose -f infra/compose.yaml up -d`, no app, no code.

Acceptance: a 3-partition topic you can `--describe`; keyed messages whose partition you can predict; a replay proving the log isn't a queue; a non-zero `LAG` you watch drain; and a rebalance you caused on purpose.

// TAKEAWAY

Kafka is just another stateful service you operate by its own CLI from inside Docker: a **topic is a log, not a queue**, partitions and keys trade global order for scale, and every group carries its own **bookmark**. You drove all of it by hand.

Tomorrow, Day 8: Quarkus Fundamentals. We start **building the application** that ties Postgres and Kafka together, and by Day 10 it will produce and consume these very topics from code, while you keep this CLI open as your debugger.